The Design and Formal Analysis of the Quantum Secure Messaging Protocol

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation

Abstract. The Quantum Secure Messaging Protocol (QSMP) is a post-quantum secure channel mechanism that supports two operational modes: a one-way authenticated *Simplex* exchange and a mutually authenticated *Duplex* exchange. Both modes combine a post-quantum key encapsulation mechanism, a post-quantum signature scheme, and a SHA3/cSHAKE key-derivation function with the RCS authenticated-encryption channel. RCS is a wide-block Rijndael-based stream cipher with a cSHAKE-based key schedule and KMAC authentication, used here as an authenticated-encryption scheme with associated data. This paper presents a formal cryptanalysis of QSMP in a multi-session adversarial model with active network control, adaptive compromise of long-term keys, and full observation of protocol transcripts.

We provide an implementation-facing engineering description of the protocol, derived from the QSMS Simplex and QSMD Duplex reference implementations and specification, and develop a symbolic protocol model suitable for cryptographic reduction proofs. For Simplex mode, we show that the client obtains unilateral authentication of the server and that the derived session keys achieve indistinguishability under compromise of the server's long-term signing key after the session. For Duplex mode, we establish mutual authentication and key indistinguishability under standard post-quantum hardness assumptions. In both modes, confidentiality and integrity of application data follow from the security of the RCS-based authenticated channel, which binds packet headers and sequence information into the associated data.

Our analysis further examines replay and reordering resistance through the authentication of sequence numbers and timestamps, evaluates the impact of long-term key compromise and ratcheting on forward secrecy, and identifies precise boundaries where security depends on the IND-CCA security of the KEM, the EUF-CMA security of the signature scheme, and the pseudorandomness and indifferentiability of SHA3 used within cSHAKE. We conclude that QSMP satisfies the stated model-dependent security goals under these assumptions, and we state explicitly where guarantees are cryptographic, where they are implementation-enforced, and where they depend on transport, deployment, or operational policy.

Keywords: QSMP · QSMS · QSMD · post-quantum secure messaging · authenticated encryption

E-mail: contact@qrcscorp.ca (John G. Underhill)

1 Introduction

1.1 Background and Motivation

The development of post-quantum secure communication systems has accelerated following the standardization of lattice-based key-encapsulation mechanisms and digital signatures, together with the growing availability of efficient symmetric constructions. Modern secure-messaging systems increasingly rely on hybrid designs that combine asymmetric and symmetric components, apply domain-separated hashing for session-key derivation, and authenticate control data to prevent transcript manipulation. The Quantum Secure Messaging Protocol (QSMP) follows this design philosophy. It provides a compact post-quantum handshake suitable for low-latency applications and builds an authenticated, encrypted data channel using the RCS wide-block stream cipher, whose key schedule and message-authentication tag are Keccak-based (cSHAKE and KMAC, respectively).

QSMP is designed for deployments where endpoints must establish secure channels in environments that may include active adversaries able to intercept, reorder, or drop packets. The protocol supports one-way authenticated operation as well as full mutual authentication. It is intended to serve as a general secure transport mechanism rather than a domain-specific protocol, so its security is expressed in a model that captures many concurrent sessions and realistic adversarial capabilities.

1.2 High-Level Description of QSMP

QSMP involves two roles, a client and a server, that exchange a sequence of structured packets containing both control information and cryptographic material. The protocol operates in two modes. *Simplex* mode provides one-way authentication: the client authenticates the server, establishes a fresh session key through a post-quantum KEM, and initializes an encrypted channel. *Duplex* mode provides mutual authentication through bidirectional exchange of signed ephemeral key material, establishment of two independent KEM-based shared secrets, and confirmation of a session-binding hash. Both modes employ SHA3 and cSHAKE for hashing and key derivation, and both use the RCS authenticated-encryption channel to protect application data. Packet headers include sequence numbers and timestamps that are authenticated as associated data.

Across both modes, QSMP maintains a stateful view of each connection containing sequence counters, expiration times, session keys, and a ratchet value. These values determine how packets are interpreted and when a session must be terminated because of invalid timestamps or protocol errors.

1.3 Security Goals and Contributions

The primary security objectives of QSMP differ between Simplex and Duplex operation. In Simplex mode the client obtains authentication of the server identity and derives a session key that remains confidential even if the server's long-term signing key is compromised after the handshake. There is no expectation of client identity authentication in this mode. In Duplex mode, both parties authenticate each other through signed ephemeral keys, and both derive session keys that achieve standard indistinguishability under passive and active adversaries. In both modes, channel confidentiality and integrity follow from the security of the RCS authenticated-encryption channel used with packet headers as associated data.

This paper provides a formal cryptanalysis of QSMP. It develops a symbolic protocol model aligned with the reference implementation, defines authentication, key indistinguishability, forward secrecy, and channel integrity in a multi-session setting, and proves that QSMP meets these goals under standard assumptions on the underlying cryptographic primitives. The analysis clarifies the scope of Simplex authentication, characterizes forward

secrecy with respect to long-term key compromise, and formalizes replay and reordering resistance through authenticated control data. The paper also examines the implications of in-band ratcheting and evaluates the overall robustness of the protocol.

1.4 Document Roadmap

Section 2 gives an engineering-level description of QSMP based directly on the reference implementation and specification, stated in implementation-independent terms. Section 3 presents the formal specification of the protocol, including roles, sessions, message flows, and adversarial capabilities. Section 4 defines authentication, key indistinguishability, forward secrecy, and channel security in the multi-session model used throughout the proofs. Section 5 states the assumptions on the cryptographic primitives. Sections 6 and 7 contain the Simplex and Duplex security proofs. Section 8 discusses the channel-binding properties of the authenticated encryption and resistance to replay and reordering. Section 9 provides a cryptanalytic evaluation. Section 10 analyzes implementation conformance and side-channel considerations. Section 11 gives concrete security estimates for recommended parameter choices. Section 12 concludes with limitations and directions for future work.

Content Index

2 Engineering Description of QSMP

This section describes the behavior of QSMP as realized by the reference implementation, using implementation-independent terminology. All details are derived from the public specification and code base and are expressed as abstract operations on state, messages, and cryptographic primitives, rather than as C-level constructs.

2.1 Roles, Trust Model, and Deployment Context

QSMP operates between two roles, a client and a server. The client initiates the connection and drives the handshake, while the server responds and enforces policy on key usage and session creation. In Duplex deployments both peers may possess long-term signing keys, but for each connection one endpoint is treated as the client role and the other as the server role within the key exchange.

The trust model assumes that server long-term signing keys are distributed to clients through an external mechanism such as certificates or preconfigured key directories. In Duplex mode each endpoint maintains a verification key for the other party and its own signing key used to authenticate its ephemeral contributions. The protocol does not define how these long-term keys are provisioned or revoked, only that they are available and that their expiration times are honored.

QSMP assumes that the post-quantum KEM provides IND-CCA security, that the signature scheme provides EUF-CMA security, and that SHA3 and cSHAKE behave as a collision-resistant hash and a pseudorandom key-derivation function within their stated domains. The RCS construction is treated as an authenticated encryption with associated data (AEAD) scheme that provides confidentiality and integrity when keyed with independent session keys.

QSMP is designed to run over a reliable, ordered transport. Packet delivery, loss, and reordering are considered under adversarial control, but the transport provides no cryptographic guarantees. Deployments may use TCP or another reliable channel that preserves byte ordering. The protocol itself enforces all cryptographic protections and imposes its own sequencing and time-validation rules.

Simplex deployments use a long-term server signing key to authenticate the server to anonymous clients that do not possess their own long-term keys. Duplex deployments suit environments where both peers maintain signing keys and seek mutual authentication and stronger binding between session keys and long-term identities.

2.2 Global Parameters and Cryptographic Primitives

QSMP is parameterized by a configuration string *cfg* that identifies the set of cryptographic algorithms and parameter choices in use. The configuration string is a fixed-length byte array that appears explicitly in handshake messages and is included in several hash computations that bind session keys to a particular algorithm suite. The specific algorithm parameter sets are not selected by QSMP itself: they are provided by the underlying QSC cryptographic library and fixed by the configuration string rather than negotiated during the handshake. The QSC default parameter selection targets NIST Category 5 (256-bit) security.

Each long-term signing key is identified by a key identity *kid*, a 16-byte array that acts as a combined device and key identifier. The identity is carried in handshake messages, is used by the server to select the appropriate verification or signing key, and is used by the client to associate a handshake with a particular expected server key.

Public verification keys and corresponding signing keys carry an expiration time represented as a 64-bit UTC value in seconds. When the current time exceeds the expiration value, the key is considered invalid and handshakes using that key must fail.

QSMP packets share a compact 21-byte header. In serialized (wire) order the header fields are:

- flag $\in \{0, \ldots, 255\}$, a one-byte message type distinguishing handshake packets, control packets, error packets, and encrypted data packets;
- msglen, a four-byte little-endian unsigned integer giving the length of the message body in bytes;
- sequence $\in \{0, \ldots, 2^{64} - 1\}$, an eight-byte little-endian unsigned integer that increases monotonically for each packet sent on a connection in a given direction;
- utctime $\in \{0, \ldots, 2^{64} - 1\}$, an eight-byte little-endian unsigned integer giving the sender's current UTC time in seconds.

Thus $[\![H]\!] = \mathsf{flag} \,\|\, \mathrm{LE32}(\mathsf{msglen}) \,\|\, \mathrm{LE64}(\mathsf{sequence}) \,\|\, \mathrm{LE64}(\mathsf{utctime})$. The header is serialized deterministically and is always treated as associated data for authenticated-encryption operations. The QSMS (Simplex) and QSMD (Duplex) implementations each bound key-exchange payloads by a protocol message maximum and enforce a fixed network receive buffer for transport-level packet acquisition. Simplex uses 32-byte hash, MAC, and symmetric-key values; Duplex uses 64-byte hash and MAC values with 512-bit symmetric keys.

QSMP instantiates the KEM and signature scheme through abstract interfaces providing key generation, encapsulation, decapsulation, signing, and verification. The active configuration pairs ML-DSA (FIPS 204, "Dilithium") with ML-KEM (FIPS 203, "Kyber"); alternative compile-time configurations pair ML-DSA with Classic McEliece and SLH-DSA (FIPS 205, "SPHINCS$^+$") with Classic McEliece. QSMS uses SHA3/cSHAKE at the 256-bit rate for transcript hashing and key derivation; QSMD uses SHA3/cSHAKE at the 512-bit rate. RCS is the authenticated stream-cipher channel for QSMP: it takes a key, a nonce, and associated data, and provides authenticated encryption and decryption for packet payloads while binding the serialized packet header into the authentication computation.

2.3 Key Material and Long-Term State

The server maintains a signing key pair (ssk, svk) together with an associated configuration string cfg, key identity kid, and expiration time. In Duplex mode each endpoint may maintain both a local signing key pair and a stored verification key for its peer.

Client-side long-term state consists of one or more peer verification keys pvk, the corresponding key identities, associated configuration strings, and expiration times. The protocol assumes that client software has verified these keys through an external mechanism prior to use.

For each handshake QSMP derives a session cookie hash sch that binds the session to long-term key material and the configuration string. In Simplex mode,

$$sch_S = \mathsf{H}(cfg \,\|\, kid \,\|\, pvk_S),$$

where pvk_S is the server verification key. Both client and server compute this value and store it as part of the handshake state. In Duplex mode the cookie includes both long-term verification keys,

$$sch_D = \mathsf{H}(cfg \,\|\, kid \,\|\, pvk_C \,\|\, pvk_S),$$

where pvk_C and pvk_S are the client and server verification keys bound by the key identity kid. This value is used as input to key derivation and is also confirmed during the Duplex establish stage.

Each connection maintains a state record that stores at least: a transmit sequence counter $txseq$ and receive sequence counter $rxseq$; the current transmit and receive RCS cipher states; the current ratchet key state $rtcs$; the last observed UTC time for the peer; and a flag indicating the last successful handshake stage. This state persists for the lifetime of a session and is erased when the session is torn down.

2.4 Handshake and Session Establishment in Simplex Mode

The Simplex handshake consists of three stages: a connect request from the client, a connect response from the server, and an exchange stage that transfers a KEM ciphertext and establishes the data channel. The implementation extends the Simplex transcript hash first with the authenticated public-key hash from the connect response and then with the asymmetric ciphertext from the exchange request, *on both the client and the server*, so that the two endpoints derive an identical transcript. The server confirms the derived transcript and keys by encrypting the final transcript hash in the exchange response.

Connect request. The client first verifies that the stored server verification key for the chosen key identity *kid* is not expired. It then constructs a message body containing *kid* and *cfg*, and assembles a packet header with the connect-request flag, the current transmit sequence number, the message-body length, and the current UTC time. The Simplex session cookie $sch_S = \mathsf{H}(cfg \parallel kid \parallel pvk_S)$ is computed and stored. The packet is then sent to the server.

Connect response. On receipt, the server deserializes the header and validates that the flag is a connect request, that the message length matches the expected size, that the sequence number is *rxseq*, and that the timestamp lies within an acceptable tolerance window relative to its local clock (default 60 seconds). It uses the received *kid* to select the appropriate server signing key and verifies that its configuration string matches *cfg*. The server computes the same cookie sch_S. It then generates an ephemeral KEM key pair (pk, sk), constructs a transcript hash over the serialized response header and the public key, *folds that hash into the running transcript*, signs the hash with its long-term signing key, and returns a connect response carrying the signature and the ephemeral public key. The server increments its transmit sequence counter after sending.

Exchange stage. The client validates the connect-response header with the same structural checks and time window. It verifies the server signature on the transcript hash and recomputes the hash over the received header and public key. If verification succeeds, the client encapsulates a shared secret to the server's ephemeral public key, obtaining a ciphertext *cpt* and secret *sec*. It folds the public-key hash and then the ciphertext into the transcript, and derives session material with cSHAKE keyed by *sec* and customized by sch_S:

$$(k_1, n_1, k_2, n_2) \leftarrow \mathsf{cSHAKE}(K = sec,\ S = sch_S).$$

These values initialize the transmit and receive RCS channels. The internal ratchet state *rtcs* is set from the Keccak state after an additional permutation, so that it does not equal any active channel key. The client sends an exchange-request packet whose body is the ciphertext *cpt* and increments its transmit sequence counter.

On receiving the exchange request, the server validates the header, decapsulates the ciphertext to recover *sec*, runs the same derivation with sch_S, initializes its RCS channels in the opposite directional roles, and stores a ratchet seed from the post-permutation Keccak state. The ephemeral KEM private key and shared secret are cleared after the RCS keys and ratchet state are initialized. The server then emits an exchange response whose body is the RCS encryption of the final transcript hash, with the serialized response header as associated data. Once the client verifies this response, the session transitions to the established state.

2.4.1 Simplex Key Exchange Pseudocode

The Simplex key exchange is implemented by `qsms_kex_simplex_client_key_exchange` and `qsms_kex_simplex_server_key_exchange` in `kex.c`, together with their internal

helpers. The pseudocode below follows the control flow of those functions, abstracting the code into a KEM-style presentation while preserving the exact cryptographic operations, header construction, and state updates. We write now for the UTC timestamp function, HeaderCreate and HeaderSerialize for the packet-header routines, H for SHA3-256, Encaps and Decaps for KEM encapsulation and decapsulation, Sign and Verify for the signature scheme, and cSHAKE for the KDF. RCS channel initialization is written RCS.Init.

Algorithm 1 Simplex.ClientConnectRequest

procedure ClientConnectRequest(kcs, cns, pkt)
  $t \leftarrow$ now()
  if $t > kcs.expiration$ **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_key_expired`
  end if
  $pkt.message \leftarrow kcs.keyid \parallel$ `QSMS_CONFIG_STRING`
  $len \leftarrow$ `KEX_SIMPLEX_CONNECT_REQUEST_MESSAGE_SIZE`
  $pkt.header \leftarrow$ HeaderCreate(`connect_request`, $cns.txseq, len$)
  $kcs.schash \leftarrow$ H(`QSMS_CONFIG_STRING` $\parallel kcs.keyid \parallel kcs.verkey$)
  $cns.exflag \leftarrow$ `connect_request`; **return** `error_none`
end procedure

Algorithm 2 Simplex.ServerConnectResponse

procedure ServerConnectResponse($kss, cns, req, resp$)
  $confs \leftarrow$ configuration bytes from $req.message$
  if KeyIdVerify($kss.keyid, req.message$) = false **then return** `error_invalid_input`
  end if
  $t \leftarrow$ now(); **if** $t > kss.expiration$ **then return** `error_key_expired`
  if $confs \neq$ `QSMS_CONFIG_STRING` **then return** `error_unknown_protocol`
  end if
  $kss.schash \leftarrow$ H(`QSMS_CONFIG_STRING` $\parallel kss.keyid \parallel kss.verkey$)
  $(kss.pubkey, kss.prikey) \leftarrow$ EncapKeyGen()
  $len \leftarrow$ `KEX_SIMPLEX_CONNECT_RESPONSE_MESSAGE_SIZE`
  $resp.header \leftarrow$ HeaderCreate(`connect_response`, $cns.txseq, len$)
  $shdr \leftarrow$ HeaderSerialize($resp.header$)
  $phash \leftarrow$ H($shdr \parallel kss.pubkey$)
  $kss.schash \leftarrow$ H($kss.schash \parallel phash$) ▷ transcript fold (matches client)
  $sig \leftarrow \mathsf{Sign}_{kss.sigkey}(phash)$
  $resp.message \leftarrow sig \parallel kss.pubkey$
  $cns.exflag \leftarrow$ `connect_response`; **return** `error_none`
end procedure

Algorithm 3 Simplex.ClientExchangeRequest

procedure ClientExchangeRequest($kcs, cns, resp, pkt$)
  parse $resp.message$ as $(sig, pubk)$
  $shdr \leftarrow$ HeaderSerialize($resp.header$); $khash \leftarrow$ H($shdr \parallel pubk$)
  if $\mathsf{Verify}_{kcs.verkey}(sig, khash)$ = false **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_authentication_failure`
  end if
  $kcs.schash \leftarrow$ H($kcs.schash \parallel khash$)
  $(ssec, cpt) \leftarrow$ Encaps($pubk$)
  $kcs.schash \leftarrow$ H($kcs.schash \parallel cpt$)
  $pkt.message \leftarrow cpt$; $len \leftarrow$ `KEX_SIMPLEX_EXCHANGE_REQUEST_MESSAGE_SIZE`
  $pkt.header \leftarrow$ HeaderCreate(`exchange_request`, $cns.txseq, len$)
  $prnd \leftarrow$ cSHAKE(K=$ssec$, S=$kcs.schash$, $outlen$=`QSC_KECCAK_256_RATE`)
  derive (k_1, n_1, k_2, n_2) from $prnd$ using the byte layout in `kex.c`
  $cns.rtcs \leftarrow$ InternalState(cSHAKE)
  RCS.Init($cns.txcpr, k_1, n_1$, true); RCS.Init($cns.rxcpr, k_2, n_2$, false)
  zeroize $ssec$ and $prnd$; $cns.exflag \leftarrow$ `exchange_request`; **return** `error_none`
end procedure

Algorithm 4 Simplex.ServerExchangeResponse

procedure ServerExchangeResponse($kss, cns, req, resp$)
  $cpt \leftarrow req.message$; $ssec \leftarrow$ Decaps($cpt, kss.prikey$)
  if decapsulation fails **then return** `error_decapsulation_failure`
  end if
  $kss.schash \leftarrow$ H($kss.schash \parallel cpt$)
  $prnd \leftarrow$ cSHAKE(K=$ssec$, S=$kss.schash$, $outlen$=`QSC_KECCAK_256_RATE`)
  derive (k_1, n_1, k_2, n_2) from $prnd$; $cns.rtcs \leftarrow$ InternalState(cSHAKE)
  RCS.Init($cns.rxcpr, k_1, n_1$, false); RCS.Init($cns.txcpr, k_2, n_2$, true)
  zeroize $ssec$ and $prnd$
  $resp.header \leftarrow$ HeaderCreate(`exchange_response`, $cns.txseq$, `KEX_SIMPLEX_EXCHANGE_RESPONSE_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($resp.header$)
  $resp.message \leftarrow$ RCS.Encrypt($cns.txcpr$, ad=$shdr$, pt=$kss.schash$)
  $cns.exflag \leftarrow$ `session_established`; **return** `error_none`
end procedure

Algorithm 5 Simplex.ClientEstablishVerify

procedure ClientEstablishVerify($kcs, cns, resp$)
  if header validation on $resp.header$ fails **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_invalid_input`
  end if
  $shdr \leftarrow$ HeaderSerialize($resp.header$)
  $sch' \leftarrow$ RCS.Decrypt($cns.rxcpr$, ad=$shdr$, ct=$resp.message$)
  if $sch' \neq kcs.schash$ **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_decryption_failure`
  end if
  $cns.exflag \leftarrow$ `session_established`; **return** `error_none`
end procedure

2.5 Handshake and Session Establishment in Duplex Mode

The Duplex handshake extends Simplex by performing mutual authentication and by deriving session keys from two independent KEM secrets. It consists of a connect stage, an exchange stage, and an establish stage that confirms the session cookie. Duplex derives its channel keys from two independent KEM shared secrets and the evolving transcript hash. The implementation supplies the first secret as the cSHAKE-512 keying string, the current 64-byte transcript hash as the customization string, and the second secret as the function-name string:

$$(k_1, n_1, k_2, n_2) \leftarrow \mathsf{cSHAKE}\big(K{=}sec_a,\ S{=}sch_D,\ N{=}sec_b\big).$$

The resulting stream is parsed into two RCS keys and two nonces. The security reduction is therefore bounded by the security of the two KEM instances, the domain separation and pseudorandomness of cSHAKE-512 (including when keying material is supplied through the function-name input; see Section 5), and the security of RCS under independent transmit and receive keys. The analysis does not treat the two KEM contributions as an additive proof of 512-bit computational security; the computational hardness of the derived keys is bounded by the stronger of the two KEM instances, and the 512-bit figure denotes the symmetric channel key length.

Connect stage. The client constructs a connect request containing *kid* and *cfg* and a signature over a hash of the serialized header together with *kid* and *cfg*. *No KEM public key is sent in the Duplex connect request.* The Duplex session cookie $sch_D = \mathsf{H}(cfg \parallel kid \parallel pvk_C \parallel pvk_S)$ is computed and stored. The server validates the header, checks that *kid* and *cfg* match a local record, verifies the client signature, and recomputes the transcript hash. If verification succeeds, the server generates an ephemeral KEM key pair and computes the same cookie. It then constructs a connect response carrying the server's ephemeral public key and a signature over a hash of the serialized response header and that public key.

Exchange stage. After validating the connect-response header and the server signature, the client encapsulates to the server's ephemeral public key to obtain sec_a (stored), generates *its own* ephemeral KEM key pair, and signs a hash binding its public key, the ciphertext cpt_a, and the serialized exchange-request header. The exchange request carries cpt_a, the client public key, and the signature. On receipt, the server verifies the client signature, decapsulates sec_a, encapsulates sec_b to the client public key (ciphertext cpt_b), derives the session keys via the cSHAKE expression above, initializes its RCS channels, and returns cpt_b with a signature over the hash of cpt_b and the serialized exchange-response header. As in Simplex, a ratchet seed *rtcs* is taken from the post-permutation Keccak state and ephemeral private keys and shared secrets are cleared after use.

Establish stage. The client decapsulates sec_b, derives the same session keys, and sends an establish request whose body is a transcript-derived confirmation value encrypted under the transmit channel with the serialized establish-request header as associated data. The server decrypts and compares the confirmation value in constant time to its own; on a match it returns an establish response whose body is the hash of that value, encrypted under its transmit channel with the establish-response header as associated data. The client decrypts, recomputes the expected hash, and compares in constant time. Any failure in decryption, verification, or comparison tears down the session and erases all key material.

2.5.1 Duplex Key Exchange Pseudocode

The Duplex key exchange is implemented by `qsmd_kex_duplex_client_key_exchange` and `qsmd_kex_duplex_server_key_exchange` in `kex.c` with internal helpers for each stage. The notation follows Section 2.4; H denotes SHA3 and cSHAKE the KDF.

Algorithm 6 Duplex.ClientConnectRequest

procedure DuplexClientConnectRequest(kcs, cns, pkt)
  $t \leftarrow$ now(); **if** $t > kcs.expiration$ **then** $cns.exflag \leftarrow$ `none`; **return** `error_key_expired`
  $pkt.message \leftarrow kcs.keyid \parallel$ `QSMD_CONFIG_STRING`
  $pkt.header \leftarrow$ HeaderCreate(`connect_request`, $cns.txseq$, `KEX_DUPLEX_CONNECT_REQUEST_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($pkt.header$)
  $phash \leftarrow$ H($shdr \parallel pkt.message$) ▷ signs header and message; no KEM key here
  append $\mathsf{Sign}_{kcs.sigkey}(phash)$ to $pkt.message$
  $kcs.schash \leftarrow$ H(`QSMD_CONFIG_STRING` $\parallel kcs.keyid \parallel kcs.verkey \parallel kcs.rverkey$)
  $cns.exflag \leftarrow$ `connect_request`; **return** `error_none`
end procedure

Algorithm 7 Duplex.ServerConnectResponse

procedure DuplexServerConnectResponse($kss, cns, req, resp$)
  $pkid \leftarrow$ key identity from $req.message$; $cfg \leftarrow$ config bytes from $req.message$
  if KeyIdLookup($pkid$) fails **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_invalid_input`
  end if
  if $cfg \neq$ `QSMD_CONFIG_STRING` **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_unknown_protocol`
  end if
  $shdr \leftarrow$ HeaderSerialize($req.header$); $phash \leftarrow$ H($shdr \parallel kid \parallel cfg$)
  if $\mathsf{Verify}_{kss.rverkey}(sig_C, phash) =$ false **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_verify_failure`
  end if
  $kss.schash \leftarrow$ H(`QSMD_CONFIG_STRING` $\parallel kss.keyid \parallel kss.rverkey \parallel kss.verkey$)
  $(kss.pubkey, kss.prikey) \leftarrow$ EncapKeyGen()
  $resp.header \leftarrow$ HeaderCreate(`connect_response`, $cns.txseq$, `KEX_DUPLEX_CONNECT_RESPONSE_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($resp.header$); $phash \leftarrow$ H($shdr \parallel kss.pubkey$)
  $resp.message \leftarrow \mathsf{Sign}_{kss.sigkey}(phash) \parallel kss.pubkey$
  $cns.exflag \leftarrow$ `connect_response`; **return** `error_none`
end procedure

Algorithm 8 Duplex.ClientExchangeRequest

procedure DUPLEXCLIENTEXCHANGEREQUEST($kcs, cns, resp, pkt$)
  parse $resp.message$ as (sig_S, pk_S); $shdr \leftarrow$ HeaderSerialize($resp.header$)
  $phash \leftarrow$ H($shdr \parallel pk_S$)
  if Verify$_{kcs.rverkey}(sig_S, phash) =$ false **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_verify_failure`
  end if
  $kcs.schash \leftarrow$ H($kcs.schash \parallel phash$)
  $(sec_a, cpt_a) \leftarrow$ Encaps(pk_S); store sec_a in kcs for later derivation
  $(kcs.pubkey, kcs.prikey) \leftarrow$ EncapKeyGen()
  $pkt.header \leftarrow$ HeaderCreate(`exchange_request`, $cns.txseq$, `KEX_DUPLEX_EXCHANGE_REQUEST_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($pkt.header$); $kch \leftarrow$ H($shdr \parallel cpt_a \parallel kcs.pubkey$)
  $kcs.schash \leftarrow$ H($kcs.schash \parallel kch$)
  $pkt.message \leftarrow cpt_a \parallel kcs.pubkey \parallel$ Sign$_{kcs.sigkey}(kch)$
  $cns.exflag \leftarrow$ `exchange_request`; **return** `error_none`
end procedure

Algorithm 9 Duplex.ServerExchangeResponse

procedure DUPLEXSERVEREXCHANGERESPONSE($kss, cns, req, resp$)
  parse $req.message$ as (cpt_a, pk_C, sig_C); $shdr \leftarrow$ HeaderSerialize($req.header$)
  $kch \leftarrow$ H($shdr \parallel cpt_a \parallel pk_C$)
  if Verify$_{kss.rverkey}(sig_C, kch) =$ false **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_verify_failure`
  end if
  $kss.schash \leftarrow$ H($kss.schash \parallel kch$)
  $sec_a \leftarrow$ Decaps($cpt_a, kss.prikey$); **if** fails **then return** `error_decapsulation_failure`
  $(sec_b, cpt_b) \leftarrow$ Encaps(pk_C)
  $resp.header \leftarrow$ HeaderCreate(`exchange_response`, $cns.txseq$, `KEX_DUPLEX_EXCHANGE_RESPONSE_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($resp.header$); $cpth \leftarrow$ H($shdr \parallel cpt_b$)
  $kss.schash \leftarrow$ H($kss.schash \parallel cpth$)
  $prnd \leftarrow$ cSHAKE(K=sec_a, S=$kss.schash$, N=sec_b, $outlen$=$3 \cdot$ `QSC_KECCAK_512_RATE`)
  derive (k_1, n_1, k_2, n_2) from $prnd$; copy post-permutation state into $cns.rtcs$
  RCS.Init($cns.rxcpr, k_1, n_1,$ false); RCS.Init($cns.txcpr, k_2, n_2,$ true)
  zeroize $sec_a, sec_b, prnd$; $resp.message \leftarrow cpt_b \parallel$ Sign$_{kss.sigkey}(cpth)$
  $cns.exflag \leftarrow$ `exchange_response`; **return** `error_none`
end procedure

Algorithm 10 Duplex.ClientEstablishRequest

procedure DUPLEXCLIENTESTABLISHREQUEST($kcs, cns, resp, pkt$)
  parse $resp.message$ as (cpt_b, sig_S); $shdr \leftarrow$ HeaderSerialize($resp.header$)
  $cpth \leftarrow$ H($shdr \parallel cpt_b$)
  if Verify$_{kcs.rverkey}(sig_S, cpth) =$ false **then**
    $cns.exflag \leftarrow$ `none`; **return** `error_verify_failure`
  end if
  $kcs.schash \leftarrow$ H($kcs.schash \parallel cpth$)
  $sec_b \leftarrow$ Decaps($cpt_b, kcs.prikey$); **if** fails **then return** `error_decapsulation_failure`
  $prnd \leftarrow$ cSHAKE(K=$kcs.ssec$, S=$kcs.schash$, N=sec_b, $outlen$=$3 \cdot$ `QSC_KECCAK_512_RATE`)
  derive (k_1, n_1, k_2, n_2) from $prnd$; copy post-permutation state into $cns.rtcs$
  RCS.Init($cns.txcpr, k_1, n_1,$ true); RCS.Init($cns.rxcpr, k_2, n_2,$ false)
  zeroize $sec_b, prnd$
  $pkt.header \leftarrow$ HeaderCreate(`establish_request`, $cns.txseq$, `KEX_DUPLEX_ESTABLISH_REQUEST_MESSAGE_SIZE`)
  $shdr \leftarrow$ HeaderSerialize($pkt.header$)
  $pkt.message \leftarrow$ RCS.Encrypt($cns.txcpr$, ad=$shdr$, pt=$kcs.schash$)
  $kcs.schash \leftarrow$ H($kcs.schash \parallel pkt.message$)
  $cns.exflag \leftarrow$ `establish_request`; **return** `error_none`
end procedure

Algorithm 11 Duplex.ServerEstablishResponse

```
procedure DuplexServerEstablishResponse(kss, cns, req, resp)
    shdr ← HeaderSerialize(req.header)
    hsch′ ← RCS.Decrypt(cns.rxcpr, ad=shdr, ct=req.message)
    if hsch′ ≠ kss.schash then
        cns.exflag ← none; return error_decryption_failure
    end if
    kss.schash ← H(kss.schash ‖ req.message)
    hhsch ← H(kss.schash)
    resp.header ← HeaderCreate(establish_response, cns.txseq, KEX_DUPLEX_ESTABLISH_RESPONSE_MESSAGE_SIZE)
    shdr_R ← HeaderSerialize(resp.header)
    resp.message ← RCS.Encrypt(cns.txcpr, ad=shdr_R, pt=hhsch)
    cns.exflag ← session_established; return error_none
end procedure
```

Algorithm 12 Duplex.ClientEstablishVerify

```
procedure DuplexClientEstablishVerify(kcs, cns, resp)
    shdr ← HeaderSerialize(resp.header)
    hhsch′ ← RCS.Decrypt(cns.rxcpr, ad=shdr, ct=resp.message)
    hhsch ← H(kcs.schash)                          ▷ constant-time comparison
    if hhsch′ ≠ hhsch then
        cns.exflag ← none; return error_decryption_failure
    end if
    cns.exflag ← session_established; return error_none
end procedure
```

2.6 Session Keys, Ratcheting, and Rekeying

In both modes the KDF is realized by cSHAKE keyed with one or two KEM secrets and customized by the session cookie. The output stream is parsed into symmetric keys and nonces for the RCS channels and an internal ratchet state *rtcs*. The ratchet state is not used directly as an encryption key; it is stored as a seed for future key derivation, taken from the Keccak state after an extra permutation so that it does not equal any active channel key.

QSMP supports two in-band rekeying mechanisms in Duplex mode, each with a dedicated packet type. The *asymmetric ratchet* (request/response flags) injects fresh KEM entropy: the initiator generates a new ephemeral KEM key pair, transmits the signed public key over the established tunnel, and the responder encapsulates a new secret whose hash is combined with the stored hash of the current session keys to re-key both directional channels. The *symmetric ratchet* (request flag) re-keys from a freshly generated random token transmitted over the encrypted channel, hashed together with the persistent ratchet state. In both cases previous RCS keys are erased once the new keys are installed, and the ratchet state is advanced one-way so that past keys cannot be reconstructed. The asymmetric ratchet additionally provides recovery from compromise of current session keys, because the new KEM secret is independent of all prior state. These two in-band exchanges are the basis of the post-compromise analysis in Section 7.

Separately, and as an optional deployment feature, the key-exchange layer can retain the peer's verification key field `rverkey` across key-exchange resets when the compile-time flag `QSMD_ASYMMETRIC_RATCHET` is enabled, allowing a higher layer to advance the peer's long-term verification key between sessions. Because `rverkey` is bound into the Duplex session cookie and used to verify connect- and exchange-stage signatures, advancing it changes the derived session keys even when other parameters are unchanged. This verification-key continuity is a key-management convenience and is distinct from the in-band ratchet exchanges that supply the post-compromise guarantee.

2.6.1 Symmetric Ratchet Seed Generation

The initial ratchet seed is generated during the handshake on both endpoints, immediately after the channel keys and nonces are derived. The construction permutes the Keccak state after extracting the channel key material, so the stored seed is independent of the active keys.

Algorithm 13 Simplex.SymmetricRatchetSeed

procedure SIMPLEXSYMMETRICRATCHETSEED($ssec, schash, cns$)
  cSHAKE.Init(K_{st}, $rate$=256, K=$ssec$, S=$schash$); securely clear $ssec$
  cSHAKE.SqueezeBlocks(K_{st}, $prnd$, 2); KeccakPermute(K_{st})
  $cns.rtcs \leftarrow K_{st}.state$[0 : QSMP_DUPLEX_SYMMETRIC_KEY_SIZE]
  derive Simplex tx/rx keys and nonces from $prnd$; securely clear $prnd$
end procedure

Algorithm 14 Duplex.SymmetricRatchetSeed

procedure DUPLEXSYMMETRICRATCHETSEED($sec_a, sec_b, schash, cns$)
  cSHAKE.Init(K_{st}, $rate$=512, K=sec_a, S=$schash$, N=sec_b); securely clear sec_a, sec_b
  cSHAKE.SqueezeBlocks(K_{st}, $prnd$, 3); KeccakPermute(K_{st})
  $cns.rtcs \leftarrow K_{st}.state$[0 : QSMP_DUPLEX_SYMMETRIC_KEY_SIZE]
  derive Duplex tx/rx keys and nonces from $prnd$; securely clear $prnd$
end procedure

2.7 Channel Protection and Packet Processing

All application data is carried in packets that use the encrypted-message flag. For each outbound packet the sender constructs the header with the appropriate flag, current sequence number, plaintext length, and current UTC time; serializes the header; sets it as associated data for the RCS cipher; encrypts the plaintext; and computes the authentication tag as part of the RCS transform. The transmit sequence counter is incremented after the packet is sent.

On reception, the peer deserializes the header and validates that the flag is an encrypted message, that the message length is within bounds, that the sequence number equals the current receive sequence counter, and that the timestamp lies within the acceptable window. It serializes the header as associated data and decrypts. If decryption and tag verification succeed, the plaintext is delivered and the receive sequence counter is incremented. Any failure in header validation or authentication results in an error and session teardown. Sequence numbers are monotonic and never reused within a session; timestamps are interpreted relative to a configured deviation window. This coupling of sequence numbers and timestamps to the AEAD associated data provides replay and reordering resistance, with the precise division of labor between the sequence counter and the AEAD binding made explicit in Section 8.

The serialized header is always supplied as associated data to the RCS AEAD, so any successful modification of these fields without detection would imply a break of RCS integrity.

2.8 Error Handling, Time Validation, and Session Teardown

QSMP defines error conditions covering invalid input, key expiration, header-validation failures, authentication failures, decapsulation failures, message-time violations, sequence mismatches, and internal allocation failures. On detecting an error a handshake function constructs an error packet carrying an error code and sends it with an error flag, after which both endpoints close the transport connection and erase all connection state, including RCS keys, ratchet state, and ephemeral secrets.

Table 1: QSMP packet header fields and their security role.

Field	Description	Security purpose
flag	Packet type (handshake step, data, keep-alive, control, or error).	Binds ciphertext to a protocol stage, preventing a ciphertext valid in one stage from being accepted in another.
sequence	Monotonic per-direction counter, incremented for each packet sent.	Provides replay and reordering detection: reused or unexpected sequence numbers are rejected by local state before AEAD.
msglen	Length of the encrypted payload as seen by the sender.	Protects framing; truncation or extension changes the authenticated header and fails the AEAD tag check.
utctime	Sender-supplied UTC time.	Enables a freshness window and detection of stale or delayed packets under the model's clock-skew assumptions.

Time validation is enforced whenever a packet is received: the timestamp is compared to local time and packets outside a configured limit are rejected. In the handshake this prevents reuse of stale messages; in the data phase it bounds the window in which replayed packets could be entertained even if their sequence numbers matched. Keep-alive behavior is an application-level policy that periodically sends minimal packets subject to the same encryption, authentication, sequence, and time rules; repeated failures or timeouts cause teardown and state erasure.

3 Formal Protocol Specification

This section formalizes QSMP as an abstract protocol executed between a client role C and a server role S. The specification is aligned with the engineering description but expressed in symbolic terms suitable for definitions and proofs. Primitives are treated as idealized interfaces and all processing steps correspond to the logical behavior of the reference implementation.

3.1 Notation and Conventions

Let $\{0,1\}^n$ denote bitstrings of length n, and $x \| y$ concatenation. For a finite set X, $x \xleftarrow{\$} X$ denotes uniform sampling. For an experiment Exp, $\Pr[\mathsf{Exp} = 1]$ is its success probability. Roles are C (client) and S (server). Each party may maintain multiple sessions indexed by a local identifier *sid*; a session is created when a party generates or receives a message starting a Simplex or Duplex handshake. Each party maintains a local clock $\mathsf{time}_P \in \mathbb{N}$, and timestamps satisfy a validity predicate $\mathsf{ValidTime}(t, \mathsf{time}_P) = 1$ iff $|t - \mathsf{time}_P|$ is below a protocol-defined threshold. A header is $H = (\mathsf{flag}, \mathsf{msglen}, \mathsf{seq}, t)$ with serialized form $[\![H]\!]$, and a packet is (H, M). The protocol references a KEM $(\mathsf{KGen}, \mathsf{Encaps}, \mathsf{Decaps})$, a signature scheme $(\mathsf{SigGen}, \mathsf{Sign}, \mathsf{Verify})$, a collision-resistant hash H, a cSHAKE-based KDF, and an AEAD channel RCS with $\mathsf{Enc}/\mathsf{Dec}$ taking associated data $AD = [\![H]\!]$.

3.2 Execution Model and Sessions

The protocol runs in a multi-session environment with a single adversary controlling all network communication; the adversary schedules activations through the oracles of Section 3.6 and may deliver, modify, drop, or reorder packets. Each party P maintains

per-session state

$$\sigma_{P,sid} = (\mathsf{mode}, txseq, rxseq, cfg, kid, pvk, \mathsf{ratchet}, k_{tx}, k_{rx}, n_{tx}, n_{rx}),$$

with $\mathsf{mode} \in \{\mathsf{Simplex}, \mathsf{Duplex}\}$, sequence counters, the bound configuration string, the peer key identity and verification key, the ratchet state, and the transmit/receive AEAD keys and nonces. Sessions begin *initiated*, become *accepted* once session keys are derived, and *established* when both parties reach acceptance.

3.3 Message Flows for Simplex Mode

The Simplex handshake is three ordered symbolic messages:

1. **Connect request.** $C \to S : (\mathsf{CR}, kid, cfg)$. The client sets $sch_S = \mathsf{H}(cfg \,\|\, kid \,\|\, pvk_S)$.
2. **Connect response.** The server generates (pk, sk), computes $h = \mathsf{H}([\![H_{resp}]\!] \,\|\, pk)$, folds h into the transcript, and replies $S \to C : (\mathsf{CRsp}, pk, \mathsf{Sign}_S(h))$. It computes the same cookie.
3. **Exchange.** The client verifies the signature, folds h then the ciphertext into the transcript, encapsulates $(cpt, sec) = \mathsf{Encaps}(pk)$, and computes $(k_{tx}, n_{tx}, k_{rx}, n_{rx}) = \mathsf{KDF}(sec, sch_S)$. It sends $C \to S : (\mathsf{EX}, cpt)$; the server decapsulates $sec = \mathsf{Decaps}(sk, cpt)$ and derives the same keys. The server confirms by encrypting the final transcript under the channel.

3.4 Message Flows for Duplex Mode

Both roles hold long-term signing keys and verify each other's ephemeral contributions. Let pvk_C, pvk_S be the long-term verification keys and define $sch_D = \mathsf{H}(cfg \,\|\, kid \,\|\, pvk_C \,\|\, pvk_S)$.

1. **Client connect request.** The client computes $h_C = \mathsf{H}([\![H_{req}]\!] \,\|\, kid \,\|\, cfg)$ and sends

   $$C \to S : (\mathsf{CR}, kid, cfg, \mathsf{Sign}_C(h_C)).$$

   No KEM public key is sent at this stage.
2. **Server connect response.** The server verifies the client signature, samples an ephemeral KEM pair (pk_S, sk_S), computes $h_S = \mathsf{H}([\![H_{resp}]\!] \,\|\, pk_S)$, and sends $S \to C : (\mathsf{CRsp}, pk_S, \mathsf{Sign}_S(h_S))$.
3. **Exchange.** The client encapsulates to pk_S to obtain the *client-to-server* secret sec_a with ciphertext cpt_a, samples its own ephemeral KEM pair (pk_C, sk_C), signs a hash over cpt_a and pk_C, and sends $C \to S : (\mathsf{EX}, cpt_a, pk_C, \mathsf{Sign}_C(\cdot))$. The server decapsulates sec_a, encapsulates to pk_C to obtain the *server-to-client* secret sec_b with ciphertext cpt_b, signs a hash over cpt_b, and returns $S \to C : (\mathsf{EXR}, cpt_b, \mathsf{Sign}_S(\cdot))$; the client decapsulates sec_b. Both parties then compute

   $$(k_{tx}, n_{tx}, k_{rx}, n_{rx}) = \mathsf{KDF}(sec_a, sch_D, sec_b),$$

   where sec_a is the client-to-server secret and sec_b the server-to-client secret, matching $\mathsf{cSHAKE}(K{=}sec_a, S{=}sch_D, N{=}sec_b)$.
4. **Establish.** With k_{tx}^C the client transmit key and k_{tx}^S the server transmit key, the client sends $C \to S : (\mathsf{EST}, \mathsf{Enc}_{k_{tx}^C}(v))$ for the transcript-derived confirmation value v; the server checks equality and returns $S \to C : (\mathsf{ESTR}, \mathsf{Enc}_{k_{tx}^S}(\mathsf{H}(v)))$. On matching, both mark the session established.

3.5 Partnering and Session Matching

Two sessions (P, sid) and (P', sid') are partners if they run in opposite roles, derive the same cookie sch, derive matching session keys (up to the role permutation of transmit and

receive), and their transcripts correspond to the same ordered handshake messages. In Simplex partnering is defined only for the client side, since the server authenticates no client identity; in Duplex partnering is symmetric.

3.6 Adversarial Interface and Oracles

The adversary controls all communication and interacts through: $\mathsf{Send}(P, sid, m)$, which delivers m and may yield a response; $\mathsf{RevealState}(P, sid)$, returning $\sigma_{P,sid}$ minus already-erased ephemeral secrets; $\mathsf{RevealKey}(P, sid)$, returning session keys of an accepted session; $\mathsf{CorruptLongTerm}(P)$, returning P's long-term signing key; $\mathsf{TamperHeader}(H)$, proposing altered header values subject to timestamp and sequence validity; and $\mathsf{Deliver}(m)$, delivering an adversary-constructed packet. The adversary cannot violate the local time-validity predicate or force acceptance of a packet with an invalid timestamp or sequence number; all AEAD decryptions use the serialized header as associated data, and decryption failures terminate the session. The Test oracle (used in key-indistinguishability) returns either the real session key or a random key of equal length under a hidden bit.

4 Security Definitions

All definitions are stated in the multi-session execution model with a single probabilistic polynomial-time adversary controlling network scheduling, delivery, and corruption. Each notion is an experiment whose advantage is the adversary's probability of causing a violation.

4.1 Server Authentication to the Client (Simplex)

In Simplex, authentication is unidirectional: only the server holds a signature key whose verification key is known to the client, and the experiment captures *the client's guarantee that it has spoken to the authentic server*. The cookie is $sch \leftarrow \mathsf{H}(cfg \parallel kid \parallel pvk_S)$. The experiment $\mathsf{Exp}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{AUTH\text{-}CLIENT}}(\mathcal{A})$ outputs 1 only if: (1) a client session (C, sid) accepts with a derived transcript, cookie, and keys; (2) no honest server session has a matching cookie and transcript; and (3) the adversary has not corrupted the server's long-term signing key before acceptance. The advantage is $\mathsf{Adv}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{AUTH\text{-}CLIENT}}(\mathcal{A}) = \Pr[\mathsf{Exp}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{AUTH\text{-}CLIENT}}(\mathcal{A}) = 1]$. There is intentionally no server-side authentication of the client in Simplex; attributing a unique client identity is outside the model.

4.2 Duplex Mutual Authentication

A session (P, sid) is accepted if it reaches the established state with derived keys and cookie sch_D. The experiment $\mathsf{Exp}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{AUTH}}(\mathcal{A})$ outputs 1 if an accepted session exists with no accepted partner at the peer sharing sch_D and matching keys (up to the role permutation), provided the adversary has not corrupted either party's long-term signing key before both sides complete their connect stages. The advantage is $\mathsf{Adv}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{AUTH}}(\mathcal{A})$ defined analogously.

4.3 Key Indistinguishability

Key indistinguishability (KI) captures confidentiality of the derived session keys against an adversary that schedules sessions, observes transcripts, and corrupts long-term keys after the handshake. The experiment $\mathsf{Exp}^{\mathsf{QSMP}}_{\mathsf{KI}}(\mathcal{A})$ proceeds: the adversary drives Simplex or Duplex handshakes; it issues at most one $\mathsf{Test}(P, sid)$ to an accepted, unrevealed session; the experiment samples $b \in \{0, 1\}$ and returns the real keys if $b = 0$ or uniform keys of

equal length if $b = 1$; the adversary outputs b'. The advantage is

$$\mathsf{Adv}^{\mathsf{QSMP}}_{\mathsf{KI}}(\mathcal{A}) = \left|\Pr[b' = b] - \tfrac{1}{2}\right|.$$

The definition applies to both modes; the source of keying material (one KEM secret in Simplex, two in Duplex) and the applicable corruption constraints differ by mode. Parameter-level entropy and the composition of the two Duplex secrets are treated conservatively in Section 11.

4.4 Forward Secrecy in Simplex and Duplex

Forward secrecy (FS) is defined relative to long-term key compromise occurring *after* a session completes. QSMP has no long-term KEM key: in Simplex the server's KEM pair is ephemeral and erased after derivation, and in Duplex both KEM pairs are ephemeral. The only long-term secret is the signing key, which never enters key derivation. The experiment $\mathsf{Exp}^{\mathsf{QSMP}}_{\mathsf{FS}}(\mathcal{A})$ lets the adversary cause (P, sid) to accept, then issue $\mathsf{CorruptLongTerm}(P)$ after acceptance, then $\mathsf{Test}(P, sid)$, with the response governed by the KI hidden-bit technique. Because signing-key corruption does not affect already-derived keys, $\mathsf{Adv}_{\mathsf{FS}} \leq \mathsf{Adv}_{\mathsf{KI}}$; the FS statement is that past session keys remain indistinguishable from random even given the later signing key.

4.5 Channel Confidentiality and Integrity

Channel security is the AEAD security of the RCS channel. Each encrypted packet is (H, C) with $AD = [\![H]\!]$, where the header carries seq and t enforcing ordering and freshness. In $\mathsf{Exp}^{\mathsf{QSMP}}_{\mathsf{CHAN\text{-}CONF}}(\mathcal{A})$ the adversary queries encryptions of chosen plaintexts under chosen headers and issues a single $\mathsf{TestChan}$, receiving a real or random ciphertext of equal length; the advantage is the usual distinguishing advantage. In $\mathsf{Exp}^{\mathsf{QSMP}}_{\mathsf{CHAN\text{-}INT}}(\mathcal{A})$ the adversary attempts a packet (H^*, C^*) such that the time predicate accepts H^*, the sequence number matches the receiver's expected value, RCS decryption succeeds, and (C^*, AD^*) was not previously produced by an honest encryption in that direction. Replay resistance derives from the monotonic sequence counter (local state); the AEAD's header binding prevents an adversary from *re-sequencing* a previously captured ciphertext, and any such accepted re-sequenced or modified packet is an INT-CTXT forgery.

4.6 Ratcheting and Post-Compromise Guarantees

For Duplex, QSMP supports in-band asymmetric and symmetric ratchets (Section 2.6). The ratchet state $\mathsf{ratchet}_i$ is a one-way evolving value derived from KDF output and any injected KEM secrets. The experiment $\mathsf{Exp}^{\mathsf{QSMP}}_{\mathsf{PCS}}(\mathcal{A})$: the adversary compromises a party's long-term signing key and observes ciphertexts under current keys; triggers an in-band ratchet update; and issues a challenge on the keys derived *after* the update. The advantage is $|\Pr[b' = b] - \frac{1}{2}|$. The definition models recovery of confidentiality after compromise, assuming fresh asymmetric or symmetric entropy is injected during ratcheting; past keys remain unrecoverable because the ratchet state evolves one-way.

5 Assumptions on Cryptographic Primitives

QSMP relies on a post-quantum KEM, a post-quantum signature scheme, a hash and extendable-output family, and the RCS authenticated channel. The assumptions below are stated for the Q1 post-quantum protocol model unless explicitly extended to Q2 oracle access. When a proof treats SHA3 or cSHAKE as a random oracle or sponge under quantum queries, that use is a separate quantum-oracle assumption.

5.1 KEM Security

The KEM $(\mathsf{KGen}, \mathsf{Encaps}, \mathsf{Decaps})$ is assumed IND-CCA secure: for any PPT $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{A}) = \left|\Pr[\mathsf{Exp}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{A}) = 1] - \frac{1}{2}\right|$ is negligible. The KEM secrets are the primary entropy fed to the KDF in both modes.

5.2 Signature Scheme Security

The signature scheme is assumed EUF-CMA secure: $\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}}(\mathcal{A}) = \Pr[\mathsf{Exp}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}}(\mathcal{A}) = 1]$ is negligible. In Simplex this prevents server impersonation; in Duplex it prevents impersonation of either party during the connect stage and guarantees both ephemeral-key-binding signatures are unforgeable.

5.3 Hash and KDF Assumptions

Collision resistance. H used for transcript hashing and cookie computation is collision resistant; $\mathsf{Adv}^{\mathsf{CR}}_{\mathsf{H}}(\mathcal{A})$ is negligible. This binds handshake fields and long-term verification keys uniquely.

Indifferentiability. The cookie *sch* is indistinguishable from random unless the adversary collides the inputs; this holds if SHA3 is treated as a random oracle or as an indifferentiable sponge.

cSHAKE as extractor and PRF. In Simplex the KDF is $\mathsf{cSHAKE}(K{=}sec, S{=}sch_S)$. In Duplex the KDF is $\mathsf{cSHAKE}(K{=}sec_a, S{=}sch_D, N{=}sec_b)$, in which a *second secret is supplied through the function-name input* N. We therefore assume cSHAKE behaves as a randomness extractor and PRF when high-min-entropy secret material is provided jointly through the keying string K and the function-name string N, with the customization string S as a public domain separator:

$$\mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{KDF}}(\mathcal{A}) = \left|\Pr[\mathcal{A}(\mathsf{KDF}(\cdot)) = 1] - \Pr[\mathcal{A}(U) = 1]\right|$$

is negligible for uniform U. This is a stronger usage assumption than the standard cSHAKE model, which treats N and S as public; an equivalent construction supplies the concatenation $sec_a \parallel sec_b$ through K. The KI and FS proofs invoke this assumption.

5.4 RCS Channel Security

RCS is modeled as an AEAD scheme and assumed to satisfy confidentiality and integrity under chosen ciphertext attack: $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{RCS}}(\mathcal{A})$ and $\mathsf{Adv}^{\mathsf{INT\text{-}CTXT}}_{\mathsf{RCS}}(\mathcal{A})$ are negligible. Because sequence numbers, timestamps, and message lengths are included in the associated data of every encrypted packet, a successful forgery subsumes re-sequencing attempts, which reduce directly to AEAD integrity. RCS is a wide-block Rijndael-based stream cipher with a cSHAKE key schedule and KMAC authentication; the AEAD abstraction is the interface relied on by the reductions.

6 Simplex Security Proofs

In Simplex the client authenticates the server, derives a session key from a single KEM secret and a binding hash of public configuration data, and initializes the encrypted channel. The server does not authenticate a client identity, so the Simplex authentication guarantee is one-sided.

6.1 Server Authentication to the Client

Theorem 1 (Simplex server authentication)**.** *For any PPT adversary $\mathcal{A}$ against Simplex client-side authentication there is an adversary $\mathcal{B}$ against the signature scheme with*

$$\mathsf{Adv}^{\mathit{QSMP\text{-}SIMPLEX}}_{\mathit{AUTH\text{-}CLIENT}}(\mathcal{A}) \le \mathsf{Adv}^{\mathit{EUF\text{-}CMA}}_{\mathit{SIG}}(\mathcal{B}) + \mathsf{negl}(\lambda).$$

Proof sketch. Suppose $\mathcal{A}$ causes the first client session (C, sid) to accept without a matching honest server partner, with the server signing key uncorrupted before acceptance. Acceptance requires a connect response carrying (pk, σ) with σ valid on $h = \mathsf{H}([\![H_{resp}]\!] \,\|\, pk)$. With no matching honest server session, σ was not produced by the honest signing algorithm on this transcript, so (h, σ) is a valid existential forgery. $\mathcal{B}$ simulates the QSMP environment for $\mathcal{A}$, answering required server signatures through its signing oracle, and outputs (h, σ) on the unmatched acceptance. Header fields and ephemeral keys entering h are honestly generated or relayed, so the forgery is nontrivial; the bound follows up to a negligible simulation-abort term. □

6.2 Key Indistinguishability in Simplex

Theorem 2 (Simplex key indistinguishability)**.** *There exist $\mathcal{B}_1, \mathcal{B}_2$ with*

$$\mathsf{Adv}^{\mathit{QSMP\text{-}SIMPLEX}}_{\mathit{KI}}(\mathcal{A}) \le \mathsf{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\mathit{PRF}}_{\mathit{KDF}}(\mathcal{B}_2) + \mathsf{negl}(\lambda).$$

Hybrid proof sketch. G_0 is the real KI experiment. In G_1 the KEM secret for the test session is replaced by an IND-CCA challenge: the reduction embeds the challenge ciphertext and one of two candidate secrets in place of $(\mathit{cpt}, \mathit{sec})$; a distinguisher yields $\mathcal{B}_1$, so $|\Pr[G_0{=}1] - \Pr[G_1{=}1]| \le \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}_1)$. In G_2 the KDF output on the challenge secret and sch_S is replaced by uniform keys; a distinguisher yields $\mathcal{B}_2$ against the KDF, so $|\Pr[G_1{=}1] - \Pr[G_2{=}1]| \le \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{KDF}}(\mathcal{B}_2)$. In G_3 the Test response is uniform; since G_2 keys are already uniform and independent, $\Pr[G_2{=}1] = \Pr[G_3{=}1]$ and the advantage in G_3 is zero. The triangle inequality gives the bound. □

6.3 Forward Secrecy for Simplex

Theorem 3 (Simplex forward secrecy)**.** *If $\mathcal{A}$ may corrupt the server signing key at any time after the target session completes, there exist $\mathcal{B}_1, \mathcal{B}_2$ with*

$$\mathsf{Adv}^{\mathit{QSMP\text{-}SIMPLEX}}_{\mathit{FS}}(\mathcal{A}) \le \mathsf{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\mathit{PRF}}_{\mathit{KDF}}(\mathcal{B}_2) + \mathsf{negl}(\lambda).$$

Proof sketch. The Simplex session key depends only on the KEM secret and the public binding hash $\mathit{sch}_S = \mathsf{H}(\mathit{cfg} \,\|\, \mathit{kid} \,\|\, \mathit{pvk}_S)$. There is no long-term KEM key; the server's KEM pair is ephemeral and erased after derivation. Once the handshake completes, the signing key never influences future computations, and post-handshake corruption of it only lets $\mathcal{A}$ forge signatures on *new* transcripts, which does not affect the already-derived key. The hybrids of Theorem 2 apply verbatim, giving the bound; equivalently $\mathsf{Adv}_{\mathsf{FS}} \le \mathsf{Adv}_{\mathsf{KI}}$. □

6.4 Discussion of Limitations in Simplex

First, authentication is strictly one-sided: the client is assured that an accepting session ran with the holder of the correct server signing key, except with probability bounded by the EUF-CMA advantage, while the server obtains no guarantee about client identity or uniqueness. Second, forward secrecy is scoped to compromise of the server signing key and depends on IND-CCA security of the KEM and pseudorandomness of the KDF; failure of those primitives could expose past keys. Third, Simplex makes no claim about client-side state misuse or application-credential compromise. Finally, the Simplex proof

in this paper does not claim post-compromise recovery after active session-key exposure. The QSMS implementation contains a symmetric in-band ratchet request path, but that path does not authenticate a new server identity or inject an authenticated fresh KEM secret; it is therefore treated as symmetric rekeying and forward evolution under the current authenticated channel, not as a full PCS theorem against endpoint or signing-key compromise. Once the server signing key is compromised before a future Simplex handshake, future sessions relying on it are open to impersonation until the verification key is replaced or revoked.

7 Duplex Security Proofs

In Duplex both parties hold long-term signing and verification keys and both contribute ephemeral KEM pairs. Session keys derive from two KEM secrets and the binding hash of the configuration string and both verification keys. Duplex thus provides mutual authentication and stronger compromise resistance than Simplex under the same primitive-level assumptions.

7.1 Mutual Authentication in Duplex

Theorem 4 (Duplex mutual authentication)**.** *There exist $\mathcal{B}_C, \mathcal{B}_S$ against the client and server signature schemes with*

$$\mathsf{Adv}^{\mathit{QSMP\text{-}DUPLEX}}_{\mathit{AUTH}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathit{EUF\text{-}CMA}}_{\mathit{SIG}}(\mathcal{B}_C) + \mathsf{Adv}^{\mathit{EUF\text{-}CMA}}_{\mathit{SIG}}(\mathcal{B}_S) + \mathsf{negl}(\lambda).$$

Proof sketch. The Duplex connect stage carries two signed messages: the client signs h_C binding the header, configuration string, and key identity, and the server signs h_S binding the header and its ephemeral public key. Consider the first accepted session with no matching partner. If it is at the client, a connect response verifies under the server key but corresponds to no run of the honest server signer, giving an EUF-CMA forgery extracted by $\mathcal{B}_S$. If it is at the server, the same applies to the client signature via $\mathcal{B}_C$. The win probability is bounded by the sum of the two EUF-CMA advantages plus negligible simulation terms. □

7.2 Key Indistinguishability in Duplex

Theorem 5 (Duplex key indistinguishability)**.** *There exist $\mathcal{B}_1, \mathcal{B}_2$ against the KEM and $\mathcal{B}_3$ against the KDF with*

$$\mathsf{Adv}^{\mathit{QSMP\text{-}DUPLEX}}_{\mathit{KI}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}}(\mathcal{B}_2) + \mathsf{Adv}^{\mathit{PRF}}_{\mathit{KDF}}(\mathcal{B}_3) + \mathsf{negl}(\lambda).$$

Hybrid proof sketch. G_0 is the real experiment. G_1 embeds an IND-CCA challenge for the client-to-server secret sec_a; G_2 embeds one for the server-to-client secret sec_b; each step bounds the gap by a KEM advantage. G_3 replaces $\mathsf{KDF}(sec_a, sch_D, sec_b)$ for the test session by uniform keys, bounded by the KDF advantage under the assumption of Section 5.3. G_4 makes the Test response uniform; since G_3 keys are already uniform and independent, $\Pr[G_3{=}1] = \Pr[G_4{=}1]$. The triangle inequality gives the stated bound. We emphasize that the two KEM secrets do not compose to an additive hardness level; each game replaces one secret, so the result holds as long as *at least one* KEM instance is IND-CCA secure, and the bound is dominated by the stronger instance. □

7.3 Forward Secrecy and Ratcheting in Duplex

Theorem 6 (Duplex forward secrecy and post-compromise recovery)**.** *Assume the KEM is IND-CCA secure, the KDF is a pseudorandom extractor (Section 5.3), and each in-band*

ratchet update incorporates fresh KEM entropy or a high-entropy symmetric token. Then there exist $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ *with*

$$\mathsf{Adv}^{\textit{QSMP-DUPLEX}}_{\textit{FS}}(\mathcal{A}) + \mathsf{Adv}^{\textit{QSMP-DUPLEX}}_{\textit{PCS}}(\mathcal{A}) \leq \mathsf{Adv}^{\textit{IND-CCA}}_{\textit{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\textit{PRF}}_{\textit{KDF}}(\mathcal{B}_2) + \mathsf{Adv}^{\textit{CR}}_{\mathsf{H}}(\mathcal{B}_3) + \mathsf{negl}(\lambda).$$

Proof sketch. Forward secrecy for the initial session follows Theorem 5: signing-key corruption after acceptance reveals neither KEM secret nor retroactively alters the KDF output, and there is no long-term KEM key. For post-compromise security, suppose the adversary learns both signing keys and current session keys at time t, and an *in-band asymmetric ratchet* executes at $t' > t$. The ratchet state $\mathsf{ratchet}_i$ is a one-way function of prior KDF outputs and injected secrets. At the update, a fresh KEM secret r (encapsulated under an uncompromised ephemeral key) is combined with $\mathsf{ratchet}_i$ and the cookie inside the KDF to produce new keys and $\mathsf{ratchet}_{i+1}$. Under IND-CCA security, r is indistinguishable from random despite past compromise; under the KDF extractor/PRF property the post-update keys are indistinguishable from random given prior keys and state; and collision resistance of H keeps the cookie binding to long-term verification keys sound. Any distinguisher for post-ratchet keys, or any use of past compromise against post-ratchet traffic, yields an adversary against the KEM, the KDF, or H, giving the bound. Recovery requires that at least one fresh high-entropy input is delivered; if every new ephemeral key is also compromised, recovery is not guaranteed. □

7.4 Replay and Reordering Resistance

Theorem 7 (Replay and reordering resistance)**.** *If RCS provides INT-CTXT security and the receiver enforces monotonic sequence numbers and valid time windows, then any adversary causing an honest party to accept a replayed or reordered application packet in Duplex yields an INT-CTXT forger:*

$$\mathsf{Adv}^{\textit{QSMP-DUPLEX}}_{\textit{CHAN-INT}}(\mathcal{A}) \leq \mathsf{Adv}^{\textit{INT-CTXT}}_{\textit{RCS}}(\mathcal{B}) + \mathsf{negl}(\lambda).$$

Proof sketch. A *pure replay* carrying an already-consumed sequence number is rejected by the monotonic counter before AEAD decryption and is not an AEAD event. For the first *accepted* replayed or reordered packet (H^*, C^*), acceptance requires that seq in H^* equals the current expected receive sequence, that the timestamp is valid, and that RCS decryption under $AD = [\![H^*]\!]$ succeeds. Since the expected sequence number advances monotonically, (C^*, AD^*) cannot equal any previously produced encryption in that direction; hence acceptance is an INT-CTXT forgery. $\mathcal{B}$ simulates honest encryptions, forwards decryptions to the RCS challenger, and outputs the accepted packet. Thus replay resistance is provided by the sequence counter, and the AEAD header binding is what prevents an adversary from re-sequencing a captured ciphertext into the currently expected slot. □

These results together establish that Duplex provides mutual authentication, key indistinguishability, forward secrecy, conditional post-compromise recovery through the in-band asymmetric ratchet, and strong channel integrity, under the stated assumptions.

8 Channel Security and AEAD Binding

The security of the QSMP data channel rests on the correct use of the RCS AEAD scheme with packet headers bound as associated data. This section formalizes that use and clarifies how replay and reordering map to AEAD events and what lies outside the cryptographic model.

8.1 AEAD Model for QSMP

For each established session the parties share a transmit key/nonce (k_{tx}, n_{tx}), a receive key/nonce (k_{rx}, n_{rx}), and sequence counters $(txseq, rxseq)$. Each application packet is (H, M) with $H = (\mathsf{flag}, \mathsf{msglen}, \mathsf{seq}, t)$ and $AD = [\![H]\!]$. The sender computes $C = \mathsf{Enc}_{k_{tx}}(n_{tx}, M, AD)$ and transmits (H, C); the receiver, after accepting H as syntactically valid and satisfying the local checks on seq and t, computes $M' = \mathsf{Dec}_{k_{rx}}(n_{rx}, C, AD)$ and accepts only on a non-error result. The model assumes that (k_{tx}, k_{rx}) is never reused across sessions, that nonces and sequence numbers are never reused per direction, and that each acceptance corresponds to a single successful AEAD decryption with the correct associated data. Under these conditions channel security reduces to the confidentiality and integrity of the AEAD scheme.

8.2 Replay and Reordering as Forgery Events

An adversary succeeds if it causes acceptance of a packet whose (C^*, AD^*) was not output by any honest encryption for that party and direction. A *pure replay* that resends an unmodified (H, C) is rejected by the sequence check before AEAD decryption, because the sequence number in H no longer matches the expected *rxseq*; such replays do not touch AEAD security and are handled by local state. A *reordering or header-tampering* attack must instead produce (H^*, C^*) such that $AD^* = [\![H^*]\!]$ passes the syntactic and time checks, seq^* equals the receiver's current *rxseq*, decryption succeeds, and (C^*, AD^*) is fresh. Any such accepted packet is an INT-CTXT forgery; the receiver's sequence and time checks rule out acceptance of any previously seen encryption.

8.3 Denial of Service and Liveness Considerations

The reductions capture confidentiality and integrity, including resistance to injected forgeries, but several availability aspects lie outside the model. The adversary may drop, delay, or reorder packets arbitrarily; the model does not guarantee that honest parties complete handshakes or deliver messages, only that an accepted packet satisfies integrity and freshness. The timestamp and sequence policies are modeled by $\mathsf{ValidTime}$ and monotonic counters, but threshold and retry choices are deployment decisions trading robustness against the replay window. Error handling on authentication failures can be abused for resource exhaustion; the analysis assumes sessions with repeated failures are torn down and that implementations bound concurrent sessions and queued connections. Liveness and DoS robustness remain transport- and deployment-level concerns.

9 Cryptanalytic Evaluation

This section examines QSMP beyond the idealized reductions: practical attack surfaces, the effect of long-term key compromise, and a high-level comparison with related protocols.

9.1 Attack Surfaces and Adversarial Capabilities

Transcript manipulation and downgrade. QSMP does not negotiate the configuration string inside the handshake; it is fixed by deployment and bound into the cookie and transcript hashes. An adversary altering *cfg* must forge signatures on altered transcripts or collide the hashes that define *sch*, both bounded by EUF-CMA and collision resistance. This removes interactive cipher-suite downgrade paths.

Key-identity confusion. The key identity *kid* maps to long-term verification keys. An adversary tricking a client into binding the wrong key to a *kid* could redirect connections;

this risk depends on external key distribution, not the protocol. Once the mapping is fixed, binding *kid* into the cookie and transcript prevents silent on-the-wire substitution.

Side-channel leakage. KEM, signature, and RCS operations are modeled as ideal black boxes; timing, cache, or power channels could leak secrets. Mitigation requires constant-time implementation and masking; such attacks fall outside the formal guarantees.

Randomness quality. QSMP depends on high-quality randomness for KEM key generation, encapsulation, signing, and ratchet updates. Weak generators could yield predictable ephemeral keys or repeated secrets, undermining the reductions; the model assumes a correctly seeded CSPRNG.

Implementation bugs and parsing. The compact binary header with length and time fields must be parsed correctly; integer overflow or missing length validation could create memory-safety or logic issues not captured symbolically. The analysis assumes correct enforcement of message sizes and rejection of unsupported flags.

9.2 Long-Term Key Compromise and Ratchet Behavior

Signing-key compromise. In Simplex, compromise of the server signing key before a handshake enables full impersonation but reveals no past KEM secrets or session keys. In Duplex, compromise of one party's signing key before its connect stage enables future impersonation but does not affect past sessions. Forward-secrecy results show past keys remain secure under the KEM and KDF assumptions.

Ephemeral KEM-key compromise. Exposure of an ephemeral KEM private key reveals only that session's secret; per-session generation and erasure localize the damage. In Duplex, exposure of one ephemeral key reveals only one of the two secrets, and the KDF input retains entropy from the other.

Session-key compromise and ratchet recovery. With an in-band asymmetric or symmetric ratchet, compromise of current session keys can be recovered once a ratchet injects fresh KEM or symmetric entropy into the KDF; subsequent keys are independent of the compromised values. Recovery depends on correct key erasure and on at least one fresh high-entropy input; persistent compromise of every new ephemeral key defeats recovery.

Cookie and transcript binding under compromise. The cookies sch_S, sch_D bind configuration strings and long-term verification keys to the KEM secrets. Compromising verification keys does not let an adversary retroactively alter a past cookie without a hash collision, which supports the Duplex establish-stage confirmation.

9.3 Comparison with Related Protocols

Relation to SSH and TLS. Traditional SSH and TLS rely on classical key exchange and signatures and often negotiate many suites. QSMP fixes a post-quantum KEM and signature scheme per configuration and avoids interactive negotiation, simplifying downgrade analysis. Its compact binary header with explicit sequence numbers and timestamps is conceptually record-oriented but tightly coupled to the AEAD associated data.

Relation to Noise and modern messaging frameworks. Noise-based protocols and systems such as Signal separate handshake patterns, key derivation, and ratcheting. QSMP shares this modular structure with explicit handshake stages, a Keccak-based KDF over multiple secrets and binding data, and a ratchet state supporting rekeying and conditional post-compromise recovery. Unlike generic frameworks, QSMP fixes a small number of flows and hard-codes the interpretation of flags and header fields, simplifying analysis at the cost of flexibility.

Post-quantum focus and assumptions. QSMP's core assumptions are IND-CCA security of a post-quantum KEM, EUF-CMA security of a post-quantum signature scheme, and well-studied properties of SHA3 and cSHAKE. The RCS channel is a wide-block Rijndael-based stream cipher with a cSHAKE key schedule and KMAC authentication, rather than a block-cipher mode layered onto classical designs. Binding both verification keys and the configuration string into the cookie and KDF inputs gives an explicit mapping from primitive-level assumptions to end-to-end guarantees. Overall, QSMP introduces no unusual structural weakness relative to established designs, provided implementations meet the constant-time and randomness requirements and key distribution correctly binds identities to verification keys.

10 Implementation Conformance and Side-Channel Considerations

This section relates the formal model to the reference implementation and states the engineering properties the model treats as idealizations.

10.1 Mapping Between Model and Reference Implementation

Handshake structure. Simplex and Duplex implement connect, exchange, and (Duplex) establish stages, each corresponding to a symbolic message flow, with explicit flag values and byte orderings.

Cookies and transcript binding. The cookies sch_S, sch_D are SHA3 hashes of configuration strings, key identities, and verification keys, computed in the same order on both endpoints. Transcript hashes for signed messages include the serialized header and the relevant ephemeral public key or ciphertext, and both endpoints fold the same values in the same order.

Key derivation and ratchet state. The KDF is cSHAKE keyed by the KEM secret(s) and customized by the cookie, with the output partitioned into RCS keys, nonces, and a ratchet seed taken from the post-permutation Keccak state, matching $\mathsf{KDF}(sec, sch_S)$ and $\mathsf{cSHAKE}(K{=}sec_a, S{=}sch_D, N{=}sec_b)$.

AEAD usage. The serialized header is supplied to RCS as associated data for both directions; decryption fails on tag mismatch or on nonce/sequence-state mismatch.

State transitions and cleanup. Ephemeral KEM private keys, shared secrets, and derived keys are erased once no longer needed, and state transitions follow the specified order so no packet is accepted out of order or with stale transcript data.

10.2 Constant-Time Requirements

The analysis assumes no secret-dependent timing leakage. The implementation must therefore run KEM decapsulation in constant time (with failures handled without revealing the cause), avoid secret-dependent branches in signature verification, perform the RCS tag comparison in constant time without leaking whether failure was early (header) or late (authentication), and use constant-time comparisons for sch_D, its hashed confirmation value, and all sensitive equality tests.

10.3 Random Number Generation and Time Synchronization

Secure randomness is required for ephemeral KEM key generation, encapsulation, signing, and ratchet entropy; low-entropy or repeated outputs could make keys predictable. QSMP uses header timestamps validated against a bounded window: loose synchronization is required, since excessive clock drift causes legitimate packets to fail ValidTime, while an overly wide window marginally enlarges the interval in which old packets pass the time check (though they are still rejected by sequence). Sequence numbers must be strictly monotonic for the lifetime of a session; the model relies on this for channel integrity.

10.4 Error Handling, Logging, and Teardown

Decryption and verification failures are terminal and erase state. On any such failure the implementation constructs an error packet, sends it if state permits, clears all cryptographic state, and tears down the connection, preventing use of keys after validation failures. Logging hooks must avoid leaking plaintext fragments, sensitive identifiers, or parsing-revealing error detail. Teardown clears RCS keys, ratchet state, ephemeral keys, and buffers, aligning with the model's erasure assumption.

11 Concrete Security Estimates

This section interprets the reductions concretely. Parameter-set selection is a property of the QSC cryptographic library, fixed by the configuration string; QSMP does not negotiate parameter strength. The QSC default targets NIST Category 5.

11.1 Parameter Choices and Security Levels

A QSMP configuration selects, through QSC, a post-quantum KEM (ML-KEM, or a code-based KEM), a post-quantum signature scheme (ML-DSA or SLH-DSA), SHA3/cSHAKE at the 256- or 512-bit rate, and the RCS channel at 256- or 512-bit key length. Security is most usefully expressed by NIST category rather than a single bit count. At the QSC default (Category 5, e.g. ML-KEM-1024 and ML-DSA-87), the asymmetric components target the Category 5 level; Simplex uses 256-bit symmetric material and Duplex uses 512-bit symmetric material. The effective security of session establishment remains bounded by the configured KEM, signature scheme, KDF, endpoint, randomness, and implementation assumptions. In Duplex, combining two KEM secrets raises the entropy supplied to the KDF and provides defense-in-depth if one contribution is weakened, but it should not be read as an additive proof of 2^{512} asymmetric hardness. The 512-bit figure for Duplex denotes the per-direction RCS channel-key length: under a generic key-search model it gives an intended 512-bit classical symmetric margin and an approximately 256-bit quantum key-search margin under Grover-style quadratic speedup assumptions.

Table 2: Security levels of QSMP primitives and dominant assumptions at the QSC Category 5 default.

Primitive	Default set	Target level	Dominant assumption
KEM	ML-KEM-1024	NIST Cat. 5	IND-CCA hardness of MLWE
Signature	ML-DSA-87	NIST Cat. 5	EUF-CMA hardness of MLWE/MSIS
Hash (Simplex)	SHA3-256	128-bit collision	Sponge indifferentiability
Hash (Duplex)	SHA3-512	256-bit collision	Sponge indifferentiability
KDF (Simplex)	cSHAKE-256	256-bit output	PRF/extractor from Keccak-p
KDF (Duplex)	cSHAKE-512	512-bit output	PRF/extractor from Keccak-p
AEAD (Simplex)	RCS-256	256-bit key	Tag strength, associated-data binding
AEAD (Duplex)	RCS-512	512-bit key	Tag strength, associated-data binding
Cookie (Simplex)	SHA3-256	128-bit collision	Collision resistance
Cookie (Duplex)	SHA3-512	256-bit collision	Collision resistance

11.2 Quantitative Bounds from the Reductions

Writing $\epsilon_{\mathsf{KEM}}, \epsilon_{\mathsf{SIG}}, \epsilon_{\mathsf{KDF}}, \epsilon_{\mathsf{RCS,conf}}, \epsilon_{\mathsf{RCS,int}}$ for the respective per-instance advantages, and ignoring negligible simulation terms, the reductions give:

$$\mathsf{Adv}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{AUTH\text{-}CLIENT}} \le \epsilon_{\mathsf{SIG}}, \qquad \mathsf{Adv}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{KI}} \le \epsilon_{\mathsf{KEM}} + \epsilon_{\mathsf{KDF}},$$

$$\mathsf{Adv}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{FS}} \le \epsilon_{\mathsf{KEM}} + \epsilon_{\mathsf{KDF}}, \qquad \mathsf{Adv}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{AUTH}} \le 2\,\epsilon_{\mathsf{SIG}},$$

$$\mathsf{Adv}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{KI}} \le 2\,\epsilon_{\mathsf{KEM}} + \epsilon_{\mathsf{KDF}}, \qquad \mathsf{Adv}^{\mathsf{QSMP\text{-}CHAN\text{-}CONF}} \le \epsilon_{\mathsf{RCS,conf}},$$

$$\mathsf{Adv}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{FS}} + \mathsf{Adv}^{\mathsf{QSMP\text{-}DUPLEX}}_{\mathsf{PCS}} \le \epsilon_{\mathsf{KEM}} + \epsilon_{\mathsf{KDF}} + \epsilon_{\mathsf{CR}}, \qquad \mathsf{Adv}^{\mathsf{QSMP\text{-}CHAN\text{-}INT}} \le \epsilon_{\mathsf{RCS,int}}.$$

The Duplex KI bound $2\,\epsilon_{\mathsf{KEM}}$ reflects two hybrid steps, not a claim that the two secrets multiply security; the guarantee holds as long as at least one KEM instance is secure. These bounds hold across many concurrent sessions provided each uses fresh KEM secrets, fresh KDF calls, and non-overlapping key material. For q sessions, standard hybrid arguments give linear scaling, e.g. $\mathsf{Adv}^{\mathsf{QSMP\text{-}SIMPLEX}}_{\mathsf{KI}}(\mathcal{A}_q) \le q\,\epsilon_{\mathsf{KEM}} + q\,\epsilon_{\mathsf{KDF}}$.

11.3 Discussion of Security Margins

KEM and signature margins. At Category 5 the asymmetric components carry substantial margin against the best known classical and quantum attacks, with no shortcuts substantially below NIST estimates; QSMP inherits these margins since session-key entropy flows from the KEM secrets.

Hash and KDF margins. SHA3 and cSHAKE provide strong indifferentiability and collision bounds; 256-bit outputs give $\approx 2^{128}$ collision resistance, ample for transcript binding and cookies, and the KDF's pseudorandomness margin is similarly strong.

AEAD margins. RCS uses a wide-block Rijndael core with increased rounds (21 for a 256-bit key, 30 for a 512-bit key), a cSHAKE key schedule, and KMAC authentication; with keys and nonces not reused and correct associated-data inputs, the channel inherits a large margin.

Composition. QSMP avoids heavy multiplicative loss by using one or two KEM secrets and a single KDF invocation per session and by avoiding symmetric-key reuse, so the overall parameter is close to that of the strongest applicable primitive rather than degraded by composition. Operational time windows and sequence bounds influence robustness, not cryptographic strength.

12 Conclusion

12.1 Summary of Results

This paper presented a formal and cryptanalytic analysis of QSMP across its Simplex and Duplex modes. It introduced an engineering-level specification aligned with the reference implementation, developed a symbolic model, and within that model defined and proved authentication, key indistinguishability, forward secrecy, channel integrity, and post-compromise recovery under standard assumptions. For Simplex we established unilateral authentication of the server based on EUF-CMA security, key indistinguishability in the multi-session model, and forward secrecy of past sessions under post-handshake signing-key compromise. For Duplex we proved mutual authentication based on both parties' signatures, key indistinguishability under adaptive adversaries, and conditional post-compromise recovery through the in-band asymmetric ratchet when fresh uncompromised KEM entropy is injected. In both modes, data-channel confidentiality and integrity reduce to the AEAD properties of RCS with the packet header as associated data, and the sequence and timestamp checks ensure that reordering and header tampering translate to INT-CTXT forgery attempts while pure replays are rejected by local state.

12.2 Limitations and Caveats

The model assumes perfect randomness, correct enforcement of monotonic sequence numbers, and bounded clock drift; failures of system entropy, clock management, or sequence handling fall outside the guarantees. Side channels are not modeled; constant-time implementation is required for the guarantees to hold in practice. External key distribution is assumed correct and authenticated out of band; certificate, provisioning, and revocation attacks are out of scope. Denial of service, connection churn, and resource exhaustion are not modeled; operational safeguards must complement the cryptographic protections. The Duplex KDF supplies a secret through the cSHAKE function-name input, which requires the usage assumption of Section 5.3; an equivalent concatenated-keying construction removes this caveat.

12.3 Directions for Future Work

Promising directions include a richer multi-stage ratchet with continuous key evolution and stronger a richer asymmetric ratchet and mechanized verification of the reference implementation's length checks, state transitions, and constant-time properties via symbolic execution or model checking; a deeper comparative study against SSH, TLS, and the Noise framework; and guidance for integrating QSMP into deployments that supply the time-synchronization, key-distribution, and randomness services its assumptions require. In summary, QSMP provides a compact, well-structured design for post-quantum secure messaging that, when implemented correctly and supported by robust system-level defenses, achieves strong authentication, confidentiality, and integrity grounded in standard assumptions on its cryptographic components.

References

[1] J. G. Underhill. *Quantum Secure Messaging Protocol (QSMP) Technical Specification.* Quantum Resistant Cryptographic Solutions Corporation, 2026.

[2] J. G. Underhill. *QSMS Simplex and QSMD Duplex Reference Implementations.* Quantum Resistant Cryptographic Solutions Corporation, 2026.

[3] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.* U.S. Department of Commerce, 2015. https://doi.org/10.6028/NIST.FIPS.202.

[4] J. Kelsey, S. Chang, and R. Perlner. *NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash.* NIST, 2016. https://doi.org/10.6028/NIST.SP.800-185.

[5] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).* U.S. Department of Commerce, 2024. https://doi.org/10.6028/NIST.FIPS.203.

[6] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).* U.S. Department of Commerce, 2024. https://doi.org/10.6028/NIST.FIPS.204.

[7] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA).* U.S. Department of Commerce, 2024. https://doi.org/10.6028/NIST.FIPS.205.

[8] D. J. Bernstein et al. *Classic McEliece: Conservative Code-Based Cryptography.* NIST PQC submission, 2022. https://classic.mceliece.org/.

[9] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. *The Keccak Reference.* Keccak Team, 2011.

[10] M. Bellare and C. Namprempre. *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm.* ASIACRYPT 2000.

[11] P. Rogaway. *Nonce-Based Symmetric Encryption.* FSE 2004.

[12] M. Bellare and P. Rogaway. *Entity Authentication and Key Distribution.* CRYPTO 1993.

[13] R. Canetti and H. Krawczyk. *Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels.* EUROCRYPT 2001.

[14] M. Abadi and P. Rogaway. *Reconciling Two Views of Cryptography.* Journal of Cryptology, 2002.